



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊

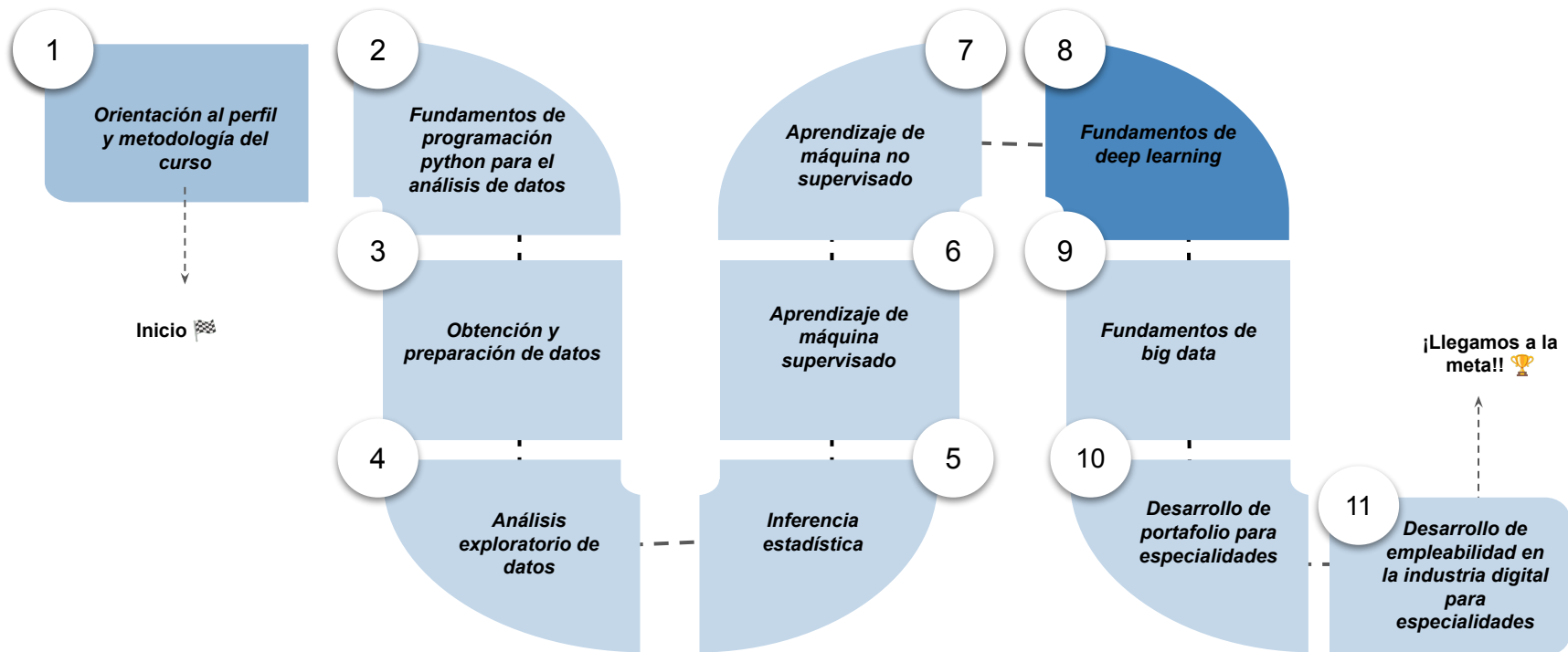


› Redes neuronales convolutivas

Aprendizaje Esperado 4: Implementar un modelo predictivo utilizando redes neuronales convolutivas para el reconocimiento de imágenes en python.

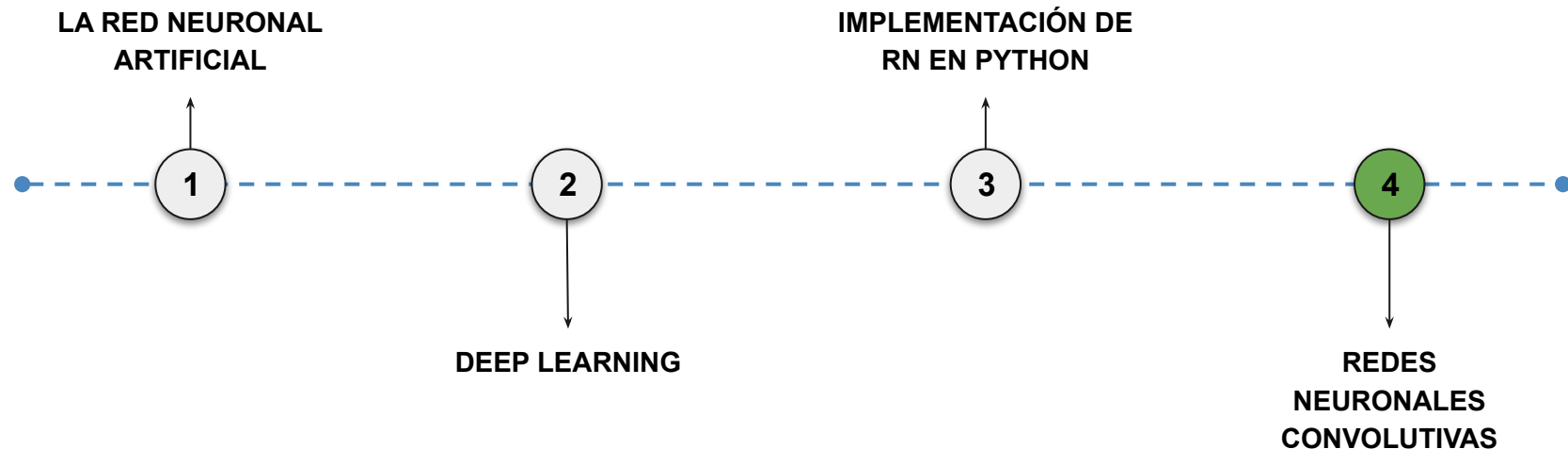
Hoja de ruta

¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



Roadmap de lecciones

¿Cuáles *lecciones* estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

1.

Fundamentos de redes neuronales convolutivas (CNN)

Introduciremos los conceptos clave de las redes neuronales convolutivas, sus componentes, ventajas y aplicaciones, preparando el terreno para implementarlas en Python.

Fundamentos y arquitectura de las CNN

¿Qué son las CNN y por qué se utilizan en procesamiento de imágenes?

Ventajas de las CNN frente a redes densas.

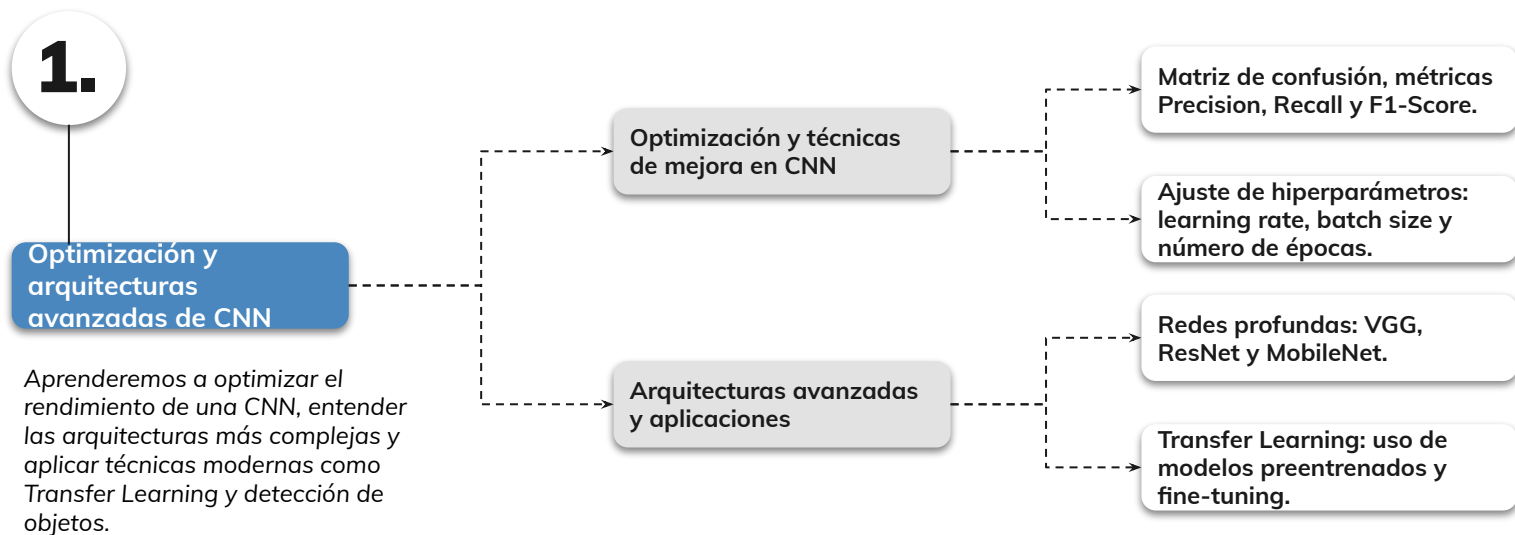
Aplicaciones y métricas en CNN

Principales casos de uso: clasificación, detección de objetos, segmentación.

Métricas para evaluar CNN en tareas de visión por computadora: accuracy, precision, recall, F1-score.

Learning Path

¿Cuáles temas trabajaremos hoy?





Objetivos de aprendizaje

¿Qué aprenderás?



- Comprender qué es una red neuronal convolutiva y sus diferencias con redes densas.
- Conocer los componentes clave de una CNN y su función dentro de la arquitectura.
- Identificar las ventajas y limitaciones de las CNN en el reconocimiento de imágenes.
- Reconocer las principales aplicaciones de las CNN en visión por computadora.
- Analizar métricas de evaluación para medir el rendimiento de una CNN.



Objetivos de aprendizaje

¿Qué aprenderás?



- Aplicar técnicas de optimización para mejorar el rendimiento de una CNN.
- Evaluar modelos usando métricas avanzadas (Precision, Recall, F1-Score).
- Conocer arquitecturas avanzadas (VGG, ResNet, MobileNet) y sus características.
- Implementar Transfer Learning y entender sus ventajas.
- Explorar aplicaciones avanzadas: detección de objetos, segmentación y GANs.

Redes neuronales convolutivas



Redes Neuronales Convolutivas: Fundamentos e Implementación

Las redes neuronales convolutivas (CNN, por sus siglas en inglés) han transformado el campo del procesamiento de imágenes y visión por computadora.

Inspiradas en el funcionamiento del sistema visual de los seres vivos, estas redes están especialmente diseñadas para reconocer patrones espaciales en imágenes mediante una combinación de operaciones matemáticas conocidas como convoluciones, funciones de activación no lineales, y reducción de dimensionalidad.

¿Qué es una red neuronal convolutiva?

Las **redes neuronales convolucionales (CNN)** son un tipo de red neuronal diseñada especialmente para trabajar con **imágenes**.

La idea es bastante intuitiva si uno lo piensa desde cómo miramos una imagen: no analizamos todos los píxeles al mismo tiempo, sino **pequeñas regiones** donde detectamos patrones (bordes, texturas, formas).

Las CNN intentan hacer algo parecido.

La idea básica

Una imagen no es más que una matriz de números.

Por ejemplo:

- Imagen en escala de grises $\rightarrow$ matriz 64×64
- Imagen RGB $\rightarrow$ tensor $64 \times 64 \times 3$

Cada número representa intensidad de color.

Una CNN **no conecta cada píxel con cada neurona** (como haría una red densa).

En lugar de eso aplica **filtros pequeños que recorren la imagen**.

A ese proceso se le llama **convolución**.

¿Qué es una convolución?

Una convolución consiste en tomar un **filtro pequeño (kernel)** y deslizarlo sobre la imagen.

Por ejemplo un filtro 3x3:

```
[ 1  0 -1  
  1  0 -1  
  1  0 -1 ]
```

Este filtro detecta **bordes verticales**.

La red aprende automáticamente estos filtros durante el entrenamiento.

Un filtro puede aprender a detectar:

- bordes
- esquinas
- texturas
- patrones complejos

Componentes de una CNN

Capas Convolutivas

Realizan operaciones convolutivas para extraer características espaciales de las imágenes.

Funciones de Activación

Introducen no linealidad al modelo, permitiendo aprender patrones complejos.

Capas de Pooling

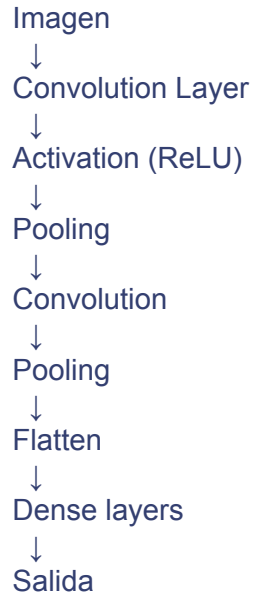
Reducen la dimensionalidad manteniendo la información más relevante.

Capas Densas

Realizan la clasificación final basada en las características extraídas.

Arquitectura típica de una CNN

Normalmente siguen esta estructura:



Cada parte tiene un rol específico.

1. Convolution Layer

Aplica varios filtros a la imagen.

Salida: **feature maps**.

Ejemplo:

imagen 64x64



32 filtros



32 mapas de características

Cada mapa detecta un patrón distinto.

2. Activación (ReLU)

Introduce **no linealidad**.

Función típica: $f(x)=\max(0,x)$

Esto permite que la red aprenda relaciones complejas.

3. Pooling

Reduce el tamaño de la imagen manteniendo la información importante.

Ejemplo: MaxPooling 2x2

$$\begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix} \rightarrow 4$$

Se queda con el máximo.

Beneficios:

- reduce dimensiones
- reduce overfitting
- hace el modelo más rápido

4. Flatten

Convierte el mapa 2D en un vector.

$$8 \times 8 \times 32 \rightarrow 2048$$

5. Dense Layers

Aquí la red se comporta como una red neuronal clásica.

Sirve para:

- clasificación
- regresión

Qué aprende una CNN

Lo interesante es que aprende **jerarquías de patrones**.

Capas tempranas detectan:

- bordes
- líneas

Capas intermedias detectan:

- texturas
- formas

Capas profundas detectan:

- ojos
- ruedas
- caras
- objetos completos

Por eso funcionan tan bien para visión artificial.

Ejemplo simple en Python

Un mini ejemplo en **TensorFlow/Keras**.

```
▶ import tensorflow as tf
   from tensorflow.keras import layers, models

   model = models.Sequential()

   model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(64,64,3)))
   model.add(layers.MaxPooling2D((2,2)))

   model.add(layers.Conv2D(64, (3,3), activation='relu'))
   model.add(layers.MaxPooling2D((2,2)))

   model.add(layers.Conv2D(64, (3,3), activation='relu'))

   model.add(layers.Flatten())

   model.add(layers.Dense(64, activation='relu'))
   model.add(layers.Dense(10, activation='softmax'))

   model.summary()
```

Esto sería una CNN para clasificar imágenes en **10 categorías**.

Dónde se usan

Las CNN aparecen en muchísimos problemas reales:

Visión artificial

- reconocimiento facial
- autos autónomos
- diagnóstico médico con radiografías

Retail

- detección de productos en estanterías

Seguridad

- detección de objetos sospechosos

Agricultura

- detección de enfermedades en plantas

Mini ejemplo visual de cómo actúa un filtro

Pensemos en una imagen pequeña en escala de grises, donde cada número representa intensidad.

Imagen 5x5

```
10 10 10 10 10
10 50 50 50 10
10 50 90 50 10
10 50 50 50 10
10 10 10 10 10
```

Ahora usamos un filtro 3x3 que detecta cambios horizontales:

```
-1 -1 -1
 0  0  0
 1  1  1
```

La CNN toma una ventana 3x3 de la imagen, multiplica elemento a elemento y suma.

Por ejemplo, sobre esta zona:

```
10 10 10
10 50 50
10 50 90
```

El cálculo sería: $(10 \cdot -1) + (10 \cdot -1) + (10 \cdot -1) + (10 \cdot 0) + (50 \cdot 0) + (50 \cdot 0) + (10 \cdot 1) + (50 \cdot 1) + (90 \cdot 1) = -10 - 10 - 10 + 0 + 0 + 0 + 10 + 50 + 90 = 120$

Ese **120** indica que en esa zona hay un cambio fuerte de intensidad.

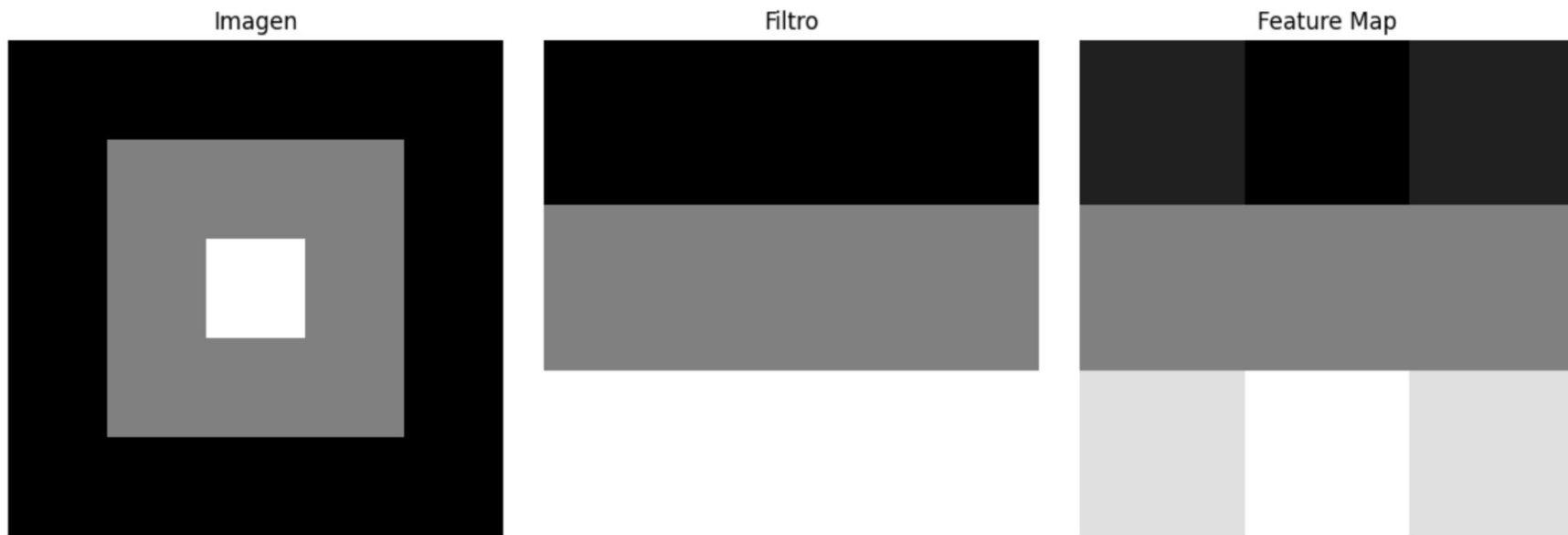
Dicho más simple: el filtro “notó” una especie de borde o transición.

Intuición

- valores altos en la salida → hay un patrón importante
- valores bajos o cercanos a 0 → no hay mucho de ese patrón

Eso produce un **mapa de características** o **feature map**.

Demo en Python de una convolución simple



Este ejemplo no entrena nada todavía. Solo muestra la mecánica de la convolución.

CNN completa con dataset real, pero no MNIST

CIFAR-10, un dataset que trae imágenes RGB de 32x32 en 10 clases:

- avión
- auto
- pájaro
- gato
- ciervo
- perro
- rana
- caballo
- barco
- camión



Ventajas de las CNN



Reducción del número de parámetros

Gracias al uso de filtros compartidos, las CNN requieren menos parámetros que las redes completamente conectadas.



Detección jerárquica de características

Permiten identificar patrones desde bajo nivel (bordes) hasta alto nivel (objetos completos).



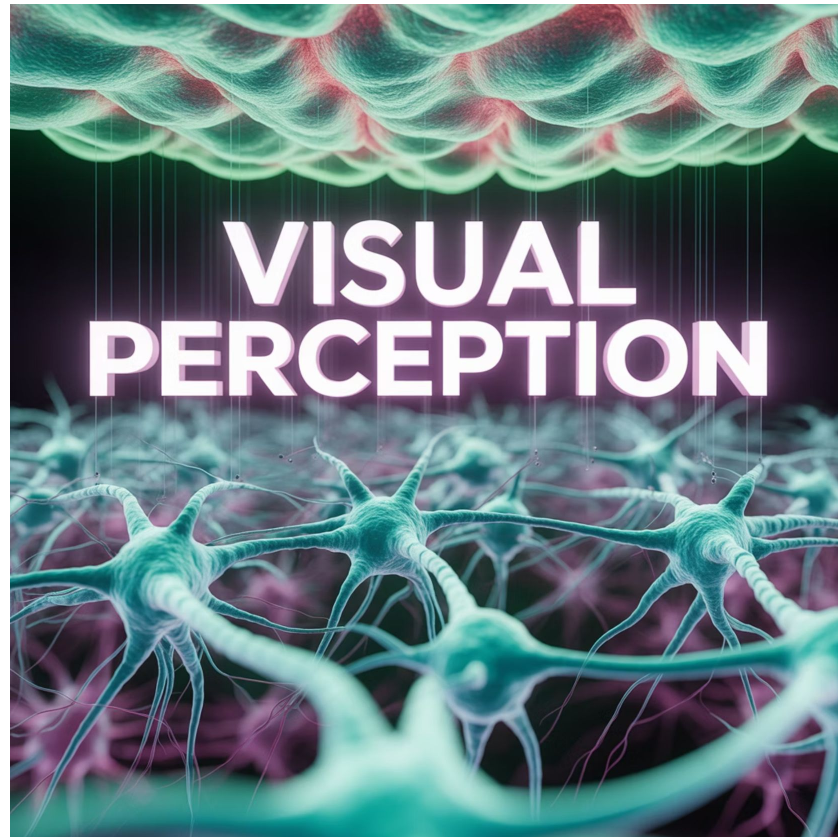
Capacidad de generalización

Funcionan bien con datos visuales complejos y variados, generalizando patrones importantes.

Inspiración Biológica

La arquitectura de las CNN se inspira en el córtex visual humano, donde ciertas neuronas responden a patrones específicos en regiones específicas del campo visual.

Esta similitud con el sistema visual biológico es una de las razones por las que las CNN son tan efectivas en tareas de procesamiento de imágenes.



¿Qué es una convolución?

La **convolución** es una operación matemática que combina dos funciones para producir una tercera. En el contexto de las CNN, se utiliza para aplicar filtros o kernels sobre una imagen, con el fin de extraer características específicas como bordes, esquinas o texturas.

Proceso de Convolución

Por ejemplo, al aplicar un filtro de 3x3 sobre una imagen, se multiplica cada valor del filtro con los valores correspondientes de un fragmento de la imagen, y luego se suman los productos. Este proceso se repite desplazando el filtro por toda la imagen (stride), generando un nuevo mapa de características.

Elementos de la Convolución

Concepto	Descripción
Imagen de entrada	Matriz de píxeles
Filtro (kernel)	Matriz pequeña (ej. 3x3 o 5x5)
Resultado	Mapa de características (feature map)

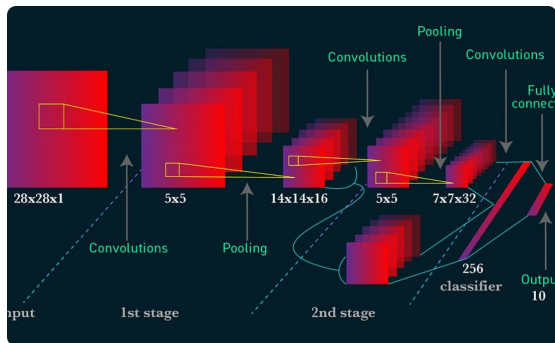
Ventajas de la Convolución

La convolución permite extraer características locales sin necesidad de aplanar la imagen completa, lo cual conserva información espacial importante. A través de múltiples capas convolutivas, se pueden detectar patrones de bajo nivel (bordes) y patrones de alto nivel (rostros, objetos).

Campo de utilización de las redes neuronales convolutivas

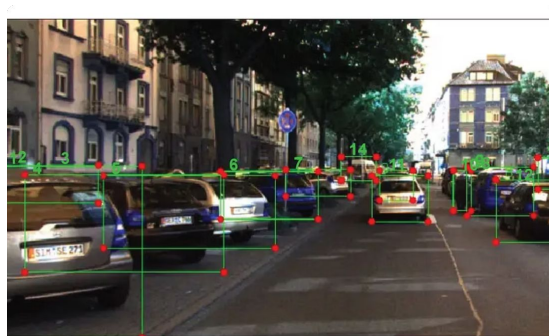
Las CNN se utilizan principalmente en tareas donde la entrada tiene una estructura espacial, siendo las imágenes el ejemplo más común.

Aplicaciones de las CNN



Clasificación de imágenes

Identificar si una imagen contiene un gato, perro, etc.



Detección de objetos

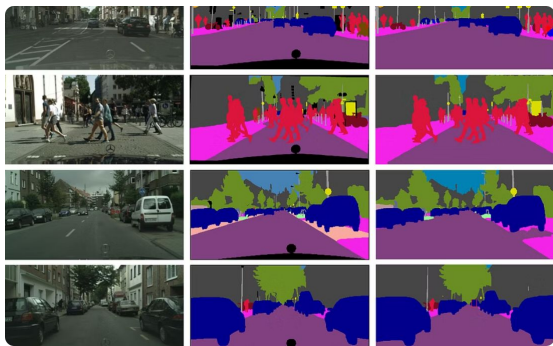
Localizar y clasificar objetos dentro de una imagen.



Reconocimiento facial

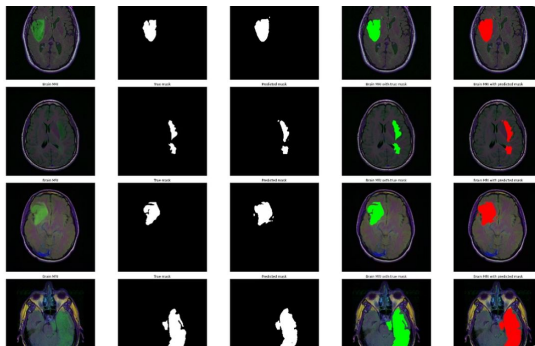
Autenticar usuarios mediante el análisis de rostros.

Más Aplicaciones de las CNN



Segmentación semántica

Clasificar cada píxel de una imagen según su clase.



Medicina

Detección de tumores en imágenes de resonancia o rayos X.



Automoción

Visión computacional para conducción autónoma.

Impacto de las CNN

Su capacidad para detectar patrones visuales ha convertido a las CNN en la arquitectura estándar en visión por computadora, reemplazando muchas técnicas tradicionales basadas en ingeniería manual de características.



Etapas en las RNC: operación de convolución y capa ReLU

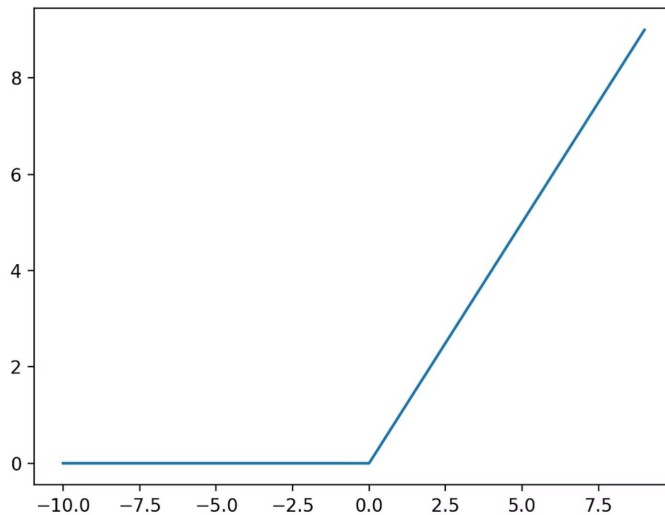
Una CNN se compone de varias etapas clave que permiten transformar progresivamente la imagen de entrada hasta generar una predicción:

1. Operación de convolución: como se explicó, aplica filtros para extraer características locales.
2. Capa ReLU (Rectified Linear Unit): introduce no linealidad, eliminando valores negativos y acelerando el entrenamiento.

Función de Activación ReLU

La fórmula de ReLU es: $f(x) = \max(0, x)$

Esta etapa permite que la red aprenda relaciones no lineales entre los datos. En conjunto, las capas de convolución y ReLU generan mapas de activación que resaltan las regiones más relevantes de la imagen.



Importancia de la Combinación Convolución + ReLU

Múltiples combinaciones de convolución + ReLU permiten detectar características cada vez más complejas, lo cual es esencial para tareas avanzadas como reconocimiento de escenas o clasificación por contexto.

Capa de pooling

El **pooling** es una operación que reduce la dimensión espacial de los mapas de activación, manteniendo las características más importantes. El tipo más común es el **Max Pooling**, que toma el valor máximo dentro de una ventana deslizante.

Beneficios del pooling

Reducción de parámetros

Disminuye significativamente la cantidad de parámetros que la red debe procesar.

Eficiencia computacional

Disminuye el tiempo de entrenamiento al reducir el tamaño de los mapas de características.

Prevención de sobreajuste

Ayuda a evitar el sobreajuste al reducir la complejidad del modelo.

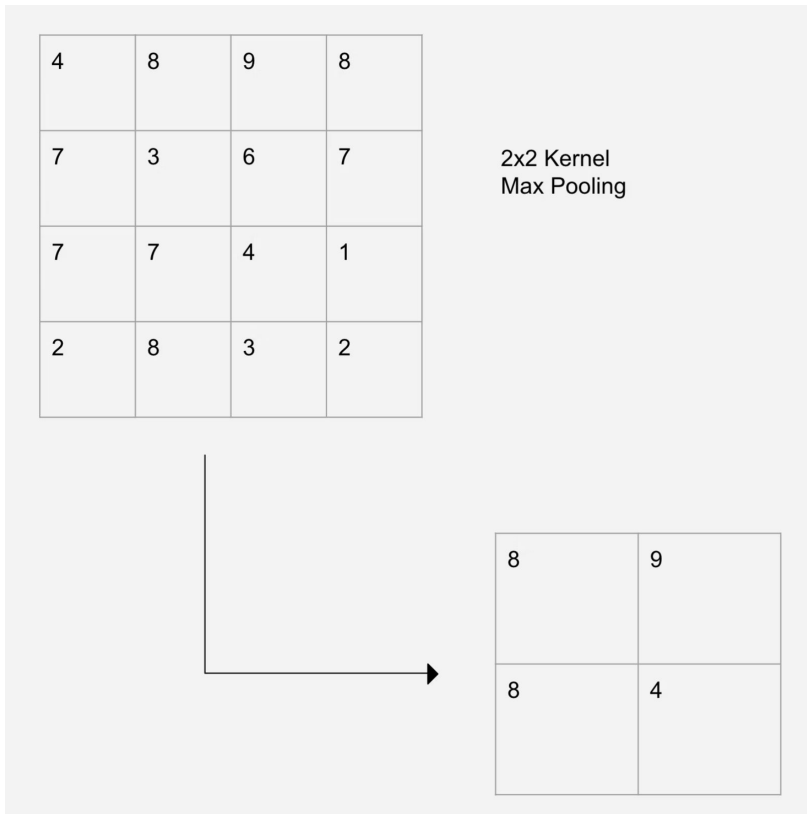
Invariancia espacial

Proporciona invariancia a pequeñas transformaciones espaciales en la imagen.

Ejemplo de Max Pooling

Por ejemplo, un Max Pooling 2x2 reduce una imagen de 32x32 a 16x16, manteniendo lo más destacado en cada región.

Esta reducción de dimensionalidad es crucial para hacer que la red sea computacionalmente eficiente mientras mantiene la información más relevante.





Flattening y Full Connection

Una vez procesadas las imágenes por capas convolutivas y de pooling, es necesario convertir los mapas de activación en un vector unidimensional. Esto se conoce como **flattening**.

Capas Fully Connected

Este vector es luego pasado a una o más capas densas (**fully connected layers**), donde cada neurona está conectada a todas las neuronas de la capa anterior. Estas capas se encargan de realizar la clasificación o regresión final.

Etapas Finales de la CNN

Etapa	Función principal
Flatten	Aplanar los mapas de activación
Full Connection	Interpretar características y generar salida

Las capas densas son similares a las usadas en redes neuronales tradicionales, y suelen ubicarse al final de las CNN como paso final de decisión.

Implementación de redes neuronales convolutivas en Python

Utilizando **Keras**, podemos construir una CNN de forma sencilla. A continuación, un ejemplo para clasificar imágenes de dígitos (dataset MNIST):

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

modelo = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D(pool_size=(2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax') # 10 clases
])
modelo.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```


Explicación del Código

1

Importación de Librerías

Se importan los módulos necesarios de Keras para construir la red neuronal.

2

Definición del Modelo

Se crea un modelo secuencial y se añaden las capas convolutivas, de pooling y densas.

3

Compilación

Se configura el optimizador, la función de pérdida y las métricas para el entrenamiento.

4

Entrenamiento y Evaluación

Se entrena el modelo con los datos y se evalúa su rendimiento.

Estructura del Modelo

Este modelo puede entrenarse con imágenes en blanco y negro de tamaño 28x28. La red extrae características espaciales, las convierte en un vector y luego predice la clase.

Evaluación y optimización de la red neuronal convolutiva

Una vez entrenada, la red debe ser evaluada con datos no vistos. Se pueden utilizar diversas métricas para medir su rendimiento.



Métricas de Evaluación

1

Accuracy

Porcentaje de predicciones correctas. Es la métrica más intuitiva pero puede ser engañosa en datasets desbalanceados.

2

Confusion Matrix

Análisis detallado por clase que muestra verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos.

3

Precision / Recall / F1-score

Métricas más robustas que consideran diferentes aspectos del rendimiento del modelo.

Live Coding

¿En qué consistirá la Demo?

Explicaremos los pasos conceptuales para implementar una red neuronal convolutiva en Python que pueda reconocer imágenes.

1. *Cómo se representan los datos de imagen (tensores RGB).*
2. *Diseño de la arquitectura CNN:*
 - a. *Capas convolutivas con filtros para extraer características.*
 - b. *Función de activación ReLU.*
 - c. *Capas de pooling para reducir dimensionalidad.*
 - d. *Capas fully connected para la clasificación final.*
3. *Elección de función de pérdida y optimizador (ej. cross-entropy y Adam).*
4. *Definición de métricas de evaluación (accuracy, precision, recall, F1).*
5. *Explicación del flujo: entrenamiento → validación → predicción.*



Ejercicio N° 1

Diseña e implementa tu primera CNN

Diseña e implementa tu primera CNN

Contexto: 🙌

Vas a aplicar los conceptos de CNN para implementar tu propio modelo de reconocimiento de imágenes en Python.

Consigna: 🛠️

1. Elige un dataset de imágenes (ej. MNIST, CIFAR-10 o similar).
2. Diseña la arquitectura de una CNN:
3. Define cuántas capas convolutivas y de pooling tendrá.
4. Selecciona funciones de activación y fully connected final.
5. Configura función de pérdida, optimizador y métricas.
6. Entrena el modelo y valida su desempeño.
7. Genera predicciones y analiza los resultados obtenidos.

Paso a paso: ⚙️

1. Prepara el dataset y normaliza las imágenes.
1. Implementa la red CNN en Python con Keras o PyTorch.
1. Evalúa el modelo en datos no vistos y analiza métricas.
1. Ajusta la arquitectura o hiperparámetros si es necesario.

Tiempo 🕒: 45 Minutos



Momento:

Time-out!



5 -10 min.



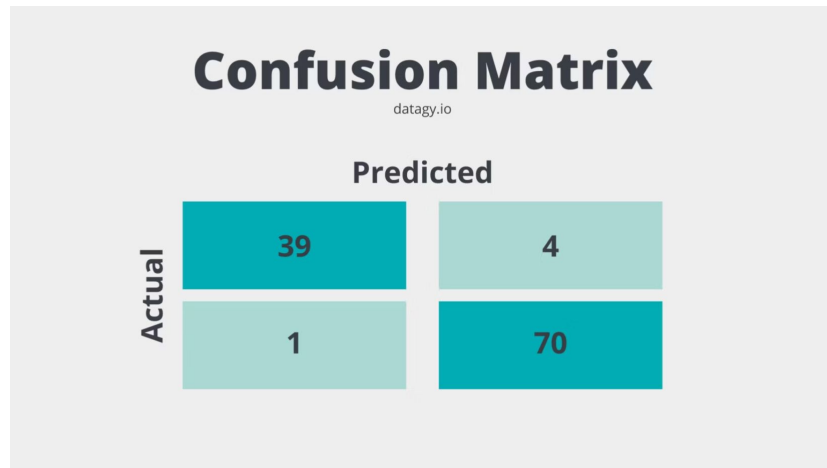
Redes neuronales convolutivas

Matriz de Confusión

La matriz de confusión es una herramienta poderosa para visualizar el rendimiento de un modelo de clasificación.

Muestra:

- Verdaderos Positivos (VP): predicciones correctas de la clase positiva
- Falsos Positivos (FP): predicciones incorrectas de la clase positiva
- Verdaderos Negativos (VN): predicciones correctas de la clase negativa
- Falsos Negativos (FN): predicciones incorrectas de la clase negativa



Precision, Recall y F1-Score

Precision

$$VP / (VP + FP)$$

Mide qué proporción de identificaciones positivas fue realmente correcta.

Recall

$$VP / (VP + FN)$$

Mide qué proporción de positivos reales fue identificada correctamente.

F1-Score

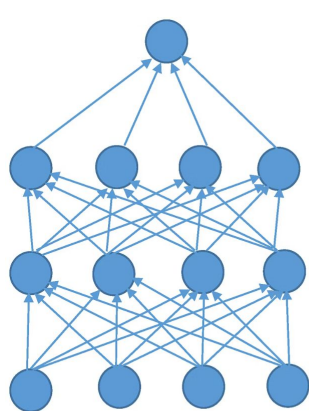
$$2 * (Precision * Recall) / (Precision + Recall)$$

Media armónica de precision y recall, útil cuando las clases están desbalanceadas.

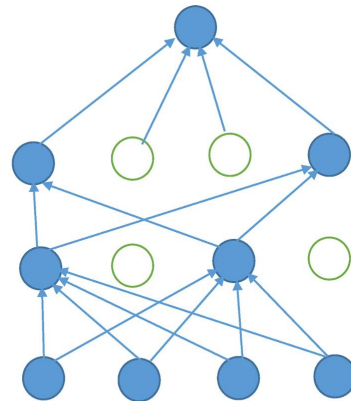
Dropout

El **Dropout** evita el sobreajuste desactivando neuronas aleatoriamente durante el entrenamiento. Esto fuerza a la red a aprender características más robustas y no depender de neuronas específicas.

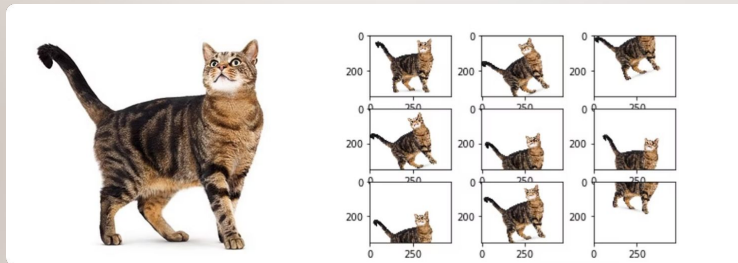
Durante la inferencia, todas las neuronas están activas pero sus pesos se escalan según la probabilidad de dropout utilizada durante el entrenamiento.



(A)



(B)



Aumento de Datos (Data Augmentation)

Data Augmentation significa **generar nuevas versiones de las imágenes de entrenamiento a partir de las existentes** mediante transformaciones.

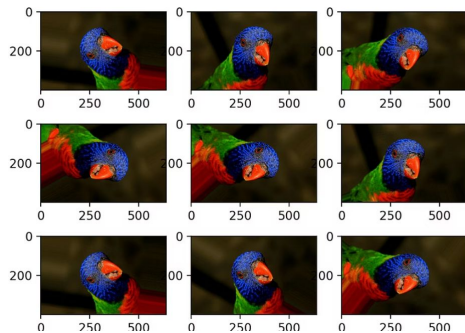
La idea es: aumentar artificialmente el tamaño del dataset sin recolectar nuevas imágenes.

Por ejemplo, si tienes una imagen de un gato, puedes generar varias variantes:

- rotarla
- moverla
- acercarla o alejarla
- reflejarla
- cambiar brillo o contraste

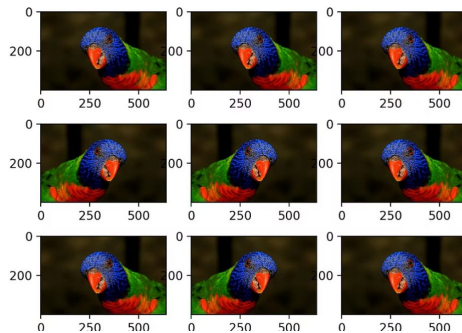
Todas siguen siendo **el mismo objeto**, pero con variaciones.

Técnicas de Aumento de Datos



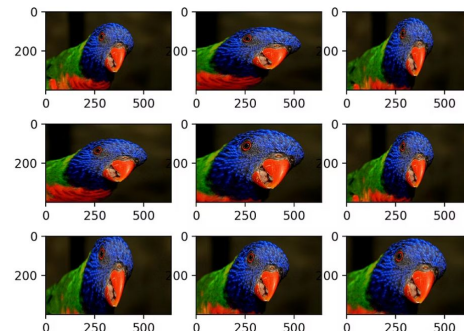
Rotación

Girar la imagen en diferentes ángulos.



Volteo

Reflejar la imagen horizontal o verticalmente.



Zoom

Acercar o alejar partes de la imagen.

En lugar de recolectar miles de imágenes adicionales, podemos generar variaciones artificiales de las imágenes existentes. Aplicando transformaciones como rotaciones, volteos o cambios de escala, el modelo aprende a reconocer el objeto independientemente de su posición, orientación o tamaño.

augmentation dentro del modelo

[]

```
▶ import tensorflow as tf
  from tensorflow.keras import layers, models

model = models.Sequential([
    tf.keras.Input(shape=(32, 32, 3)),

    # Data Augmentation
    layers.RandomRotation(0.1),          # rota aprox. ±10%
    layers.RandomFlip("horizontal"),     # volteo horizontal
    layers.RandomZoom(0.1),              # zoom in/out

    # CNN
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(128, (3, 3), activation='relu'),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```


¿Cuándo utilizarla?

Situación	Usar augmentation
Dataset pequeño	Sí
Overfitting	Sí
Objetos con variaciones de posición	Sí
Dataset muy grande y diverso	A veces no necesario

El aumento de datos se utiliza principalmente cuando el modelo presenta sobreajuste o cuando el dataset es pequeño, ya que introduce variaciones artificiales que ayudan a la red a aprender patrones más generales.

Cuando aparece overfitting

La señal más común es esta.

Durante el entrenamiento ves algo así:

Epoch	Accuracy train	Accuracy validation
1	0.65	0.60
5	0.85	0.66
10	0.95	0.67

La red aprende muy bien el **training**, pero no mejora en **validation**.

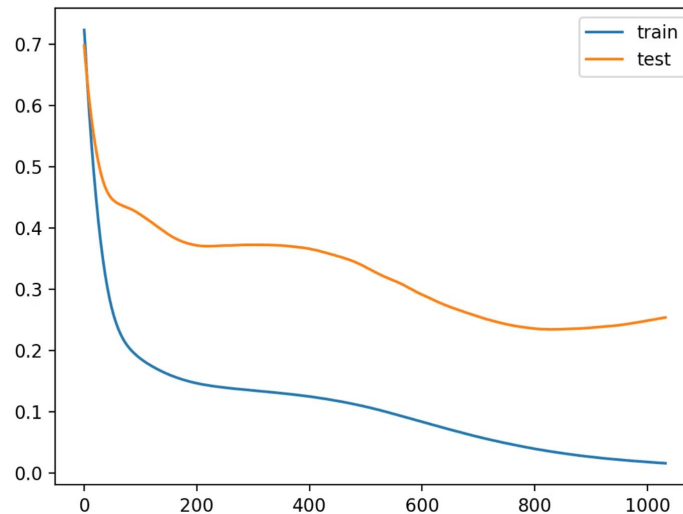
Esto significa que el modelo está **memorizando las imágenes** en lugar de aprender patrones generales.

Ahí el augmentation ayuda porque introduce variaciones.

Early Stopping

El **Early Stopping** detiene el entrenamiento si la red deja de mejorar en el conjunto de validación durante un número determinado de épocas.

Esto evita el sobreajuste y ahorra tiempo de entrenamiento.



Curva	Qué representa
train (azul)	error en entrenamiento
validation (naranja)	error en validación

Early Stopping

Cuándo se usa Early Stopping

Se usa cuando:

- 1. El modelo empieza a sobreajustar:** El modelo sigue mejorando en train pero empeora en validation. Ahí Early Stopping detiene el entrenamiento.
- 2. No sabes cuántas epochs entrenar:** En deep learning muchas veces no sabes si entrenar. Early stopping decide automáticamente cuándo parar.
- 3. Para ahorrar tiempo de entrenamiento:** En redes grandes el entrenamiento puede durar horas o días. Early stopping evita entrenar más de lo necesario.

Early Stopping

Cómo funciona internamente

El algoritmo es simple.

Se define un parámetro llamado **patience**.

Ejemplo:

`patience = 5`

Significa: si durante 5 epochs seguidas el validation loss no mejora → detener entrenamiento.

Cómo usarlo en Keras

Se usa con un **callback**.

EarlyStopping

[]

```
▶ from tensorflow.keras.callbacks import EarlyStopping
```

```
early_stop = EarlyStopping(  
    monitor='val_loss',  
    patience=5,  
    restore_best_weights=True  
)
```

[]

```
history = model.fit(  
    x_train, y_train,  
    epochs=50,  
    batch_size=64,  
    validation_split=0.2,  
    callbacks=[early_stop]  
)
```

Ajuste de Hiperparámetros

Learning Rate

Controla el tamaño de los pasos durante la optimización. Un valor demasiado grande puede causar divergencia, mientras que uno demasiado pequeño puede hacer que el entrenamiento sea muy lento.

Tamaño de Lotes (Batch Size)

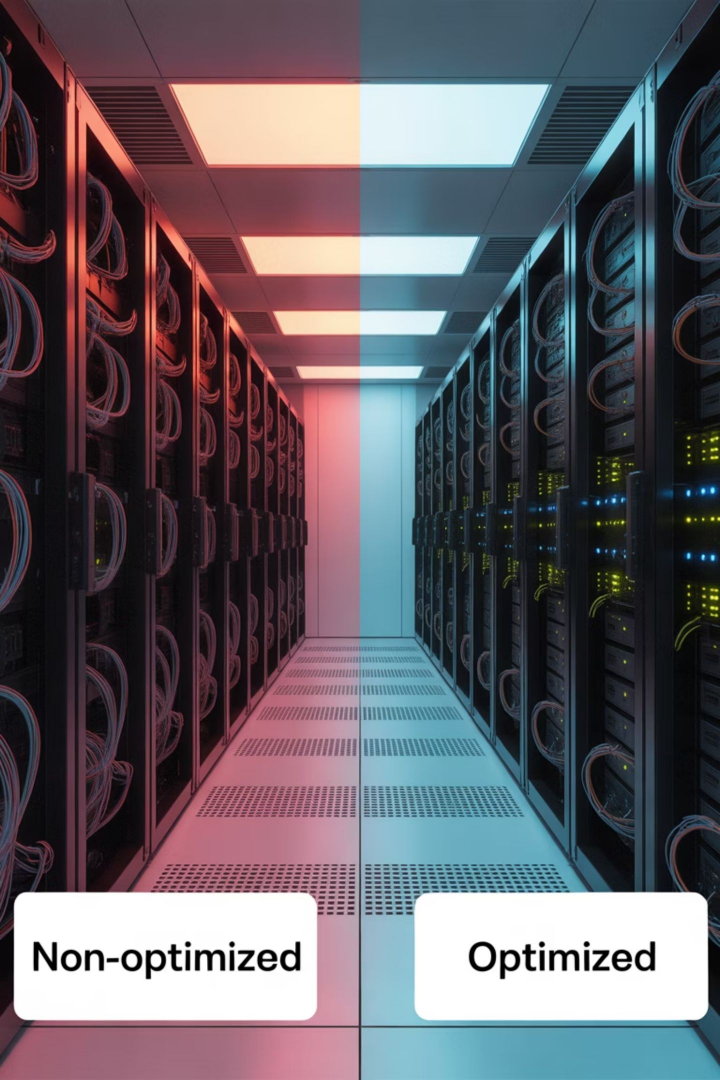
Determina cuántas muestras se procesan antes de actualizar los pesos. Afecta tanto la velocidad de entrenamiento como la calidad de la convergencia.

Número de Épocas

Cuántas veces se procesa todo el conjunto de datos. Demasiadas épocas pueden llevar al sobreajuste.

Arquitectura de la Red

Número y tamaño de las capas, tipos de activación, etc. La arquitectura óptima depende del problema específico.



Impacto de las Técnicas de Optimización

Estas técnicas mejoran la capacidad de generalización del modelo, haciéndolo útil para trabajar con datos reales y variados.

Non-optimized

Optimized

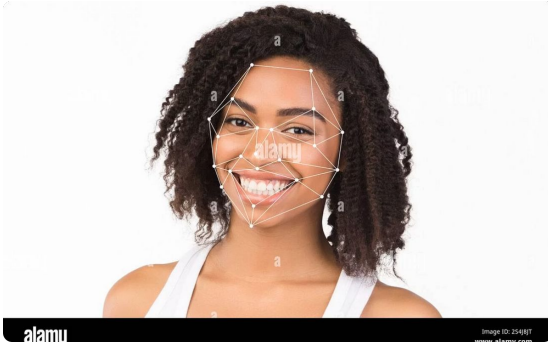
Conclusiones sobre las CNN

Las redes neuronales convolutivas son la herramienta por excelencia para abordar problemas de reconocimiento y clasificación de imágenes. Su arquitectura permite extraer patrones espaciales y jerárquicos de forma eficiente, logrando resultados superiores a métodos tradicionales.

Aplicación del Conocimiento

El conocimiento adquirido te permitirá no solo construir modelos funcionales para tareas de visión por computadora, sino también evaluar su rendimiento y mejorarlos con técnicas de regularización y optimización.

Importancia de las CNN en la Tecnología Actual



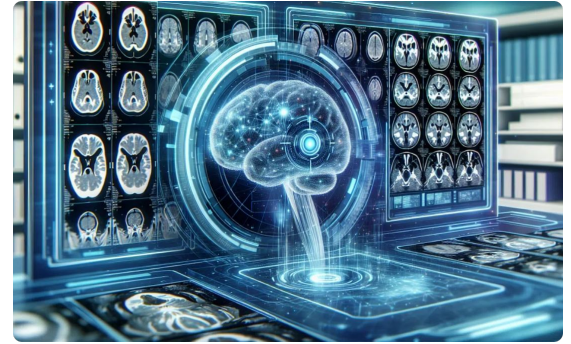
Reconocimiento Facial

Sistemas de seguridad y autenticación basados en el análisis de rostros.



Conducción Autónoma

Sistemas de percepción visual para vehículos sin conductor.



Diagnóstico por Imagen

Detección automática de patologías en imágenes médicas.



Habilidades Clave

Las CNN son la base de tecnologías como el reconocimiento facial, la conducción autónoma y el diagnóstico por imagen, por lo que dominarlas es una habilidad clave para cualquier profesional del aprendizaje automático.

Próximos Pasos

Dominar los Fundamentos

Comprender a fondo los conceptos básicos de las CNN presentados en este manual.

Explorar Arquitecturas Avanzadas

Investigar modelos más complejos como VGG, ResNet o MobileNet.

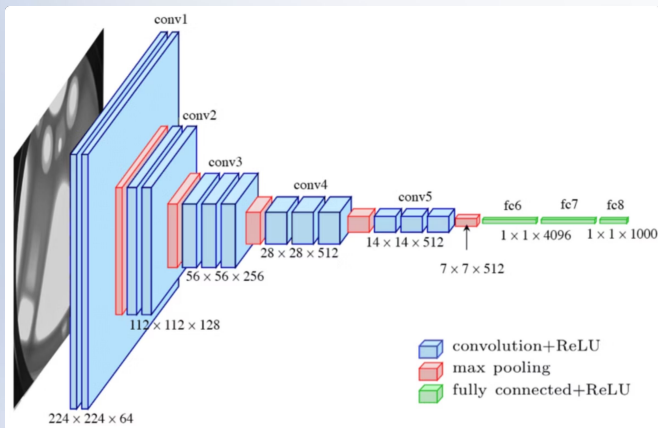
Experimentar con Implementaciones

Practicar con el código de ejemplo y adaptarlo a diferentes conjuntos de datos.

Aplicar a Proyectos Reales

Utilizar lo aprendido en aplicaciones prácticas y proyectos del mundo real.

Arquitecturas Avanzadas



A medida que los problemas de visión por computador se volvieron más complejos, las redes convolucionales también tuvieron que evolucionar. Las arquitecturas avanzadas son diseños de redes neuronales que optimizan cómo se extraen características de las imágenes, mejorando la precisión, la estabilidad del entrenamiento o la eficiencia computacional.

Estas arquitecturas ya no son redes pequeñas como la que usamos con CIFAR-10.

Muchas de ellas tienen **decenas o incluso cientos de capas**.

Las más conocidas son:

- **VGG**
- **ResNet**
- **MobileNet**
- Inception
- EfficientNet

Arquitectura VGG

VGG es una arquitectura CNN desarrollada por el Visual Geometry Group de Oxford.

Se caracteriza por:

- Uso de filtros pequeños (3x3) en todas las capas convolutivas
- Múltiples capas convolutivas seguidas antes del pooling
- Incremento gradual del número de filtros en capas más profundas

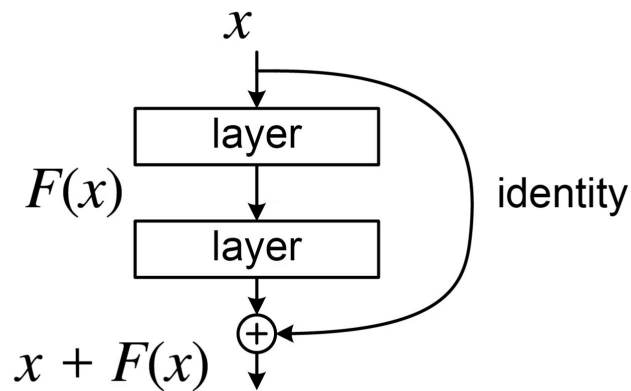
Es conocida por su simplicidad y profundidad, aunque requiere muchos parámetros.

Arquitectura ResNet

ResNet (Residual Network) introdujo las conexiones residuales para solucionar el problema de la degradación en redes muy profundas.

Características principales:

- Conexiones de salto (skip connections) que permiten que la información fluya directamente a capas posteriores
- Capacidad para entrenar redes extremadamente profundas (50, 101, 152 capas)
- Mejor gradiente de flujo durante el entrenamiento



Arquitectura MobileNet

MobileNet está diseñada específicamente para aplicaciones móviles y dispositivos con recursos limitados:

- Uso de convoluciones separables en profundidad para reducir parámetros
- Balance entre precisión y eficiencia computacional
- Hiperparámetros para ajustar el tamaño y velocidad del modelo

Arquitectura	Idea principal	Ventaja
VGG	muchos filtros 3×3	arquitectura simple y profunda
ResNet	conexiones residuales	permite redes muy profundas
MobileNet	convoluciones separables	eficiente para móviles

En vez de entrenar una CNN grande desde cero, podemos cargar una arquitectura conocida como VGG o ResNet con pesos preentrenados en ImageNet y reutilizarla. (**transfer learning**.)

Ejemplo simple con VGG16

```
▶ import tensorflow as tf
   from tensorflow.keras.applications import VGG16
   from tensorflow.keras import layers, models

   base_model = VGG16(
       weights='imagenet',
       include_top=False,
       input_shape=(224, 224, 3)
   )

   base_model.trainable = False # congelamos la base convolucional

   model = models.Sequential([
       base_model,
       layers.Flatten(),
       layers.Dense(128, activation='relu'),
       layers.Dense(10, activation='softmax')
   ])

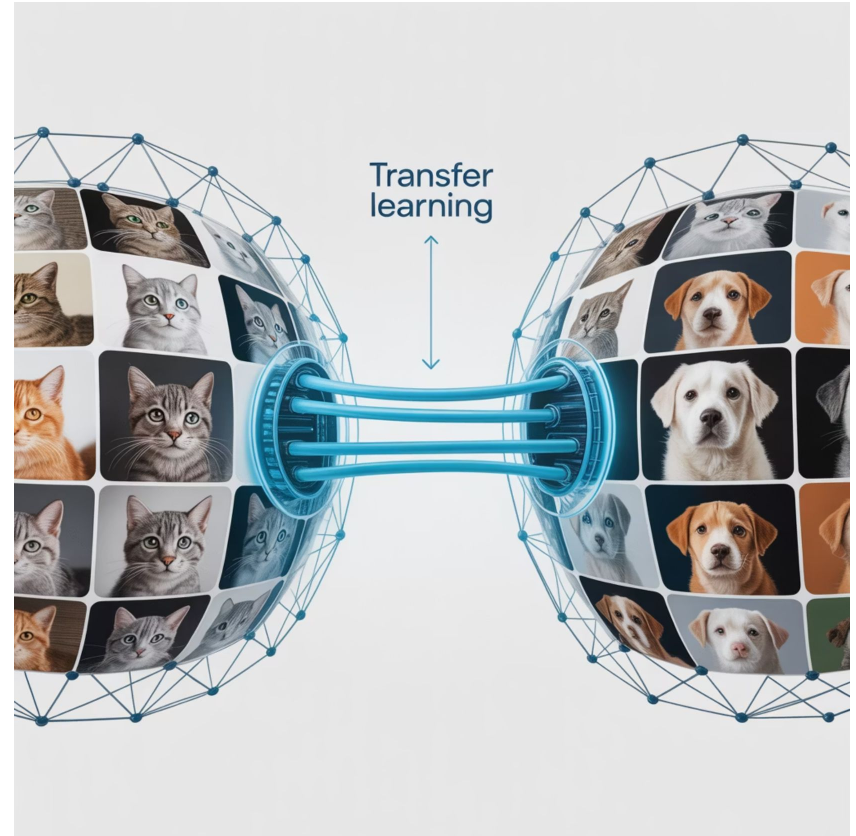
   model.summary()
```

Un modelo preentrenado es una red que ya fue entrenada con millones de imágenes y que puede reutilizarse para nuevos problemas. En lugar de aprender desde cero, aprovechamos las características visuales que ya conoce y solo adaptamos la última parte a nuestras clases.

Transfer Learning

El Transfer Learning permite aprovechar modelos preentrenados en grandes conjuntos de datos para aplicarlos a problemas específicos con menos datos:

- Utiliza los pesos de un modelo entrenado en millones de imágenes
- Reemplaza solo las últimas capas para adaptarlo a nuevas clases
- Requiere mucho menos datos de entrenamiento
- Reduce significativamente el tiempo de entrenamiento



Aplicación de Transfer Learning

1

Seleccionar Modelo Base

Elegir una arquitectura preentrenada como VGG16, ResNet50 o MobileNet.

2

Congelar Capas

Mantener fijos los pesos de las capas convolutivas que extraen características generales.

3

Añadir Nuevas Capas

Reemplazar las capas densas finales con nuevas capas adaptadas a tu problema específico.

4

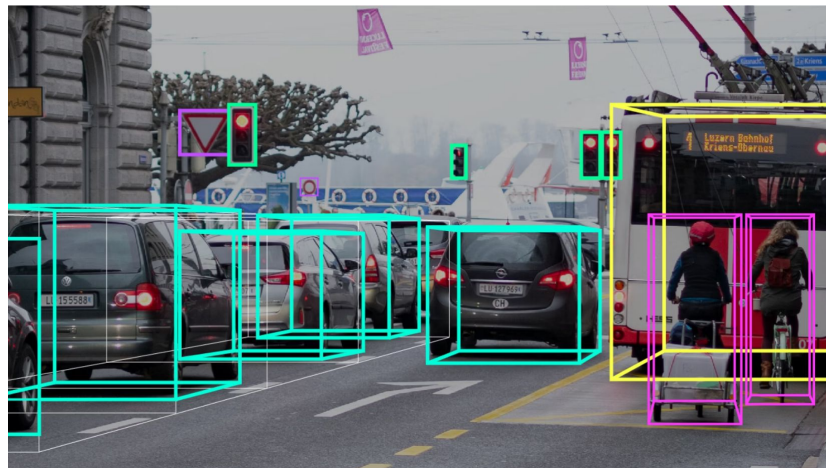
Fine-Tuning

Opcionalmente, entrenar algunas de las capas superiores del modelo base para adaptarlas mejor a los nuevos datos.

Detección de Objetos con CNN

Hasta ahora vimos modelos que clasifican imágenes completas. Sin embargo, muchos problemas reales requieren identificar múltiples objetos y su ubicación. Para esto existen modelos de detección de objetos como R-CNN, YOLO y SSD, que permiten identificar y localizar varios objetos dentro de una imagen.

- R-CNN, Fast R-CNN y Faster R-CNN: utilizan propuestas de regiones
- YOLO (You Only Look Once): enfoque de una sola pasada, más rápido
- SSD (Single Shot Detector): balance entre velocidad y precisión



Segmentación Semántica

La segmentación semántica clasifica cada píxel de la imagen en una categoría específica.

No solo detecta objetos, sino que pinta toda la región donde está el objeto.



azul = cielo
gris = carretera
rojo = persona
verde = árboles
amarillo = autos

Cada píxel pertenece a una clase.

Tarea	Qué hace
Clasificación	dice qué hay en la imagen
Detección	dibuja cajas alrededor de objetos
Segmentación	clasifica cada píxel

Arquitecturas más usadas.

- U-Net: arquitectura en forma de U con conexiones de salto
- SegNet: codificador-decodificador con índices de pooling
- DeepLab: utiliza convoluciones atrous para campos receptivos más grandes

Es crucial en aplicaciones médicas, conducción autónoma y análisis de imágenes satelitales.

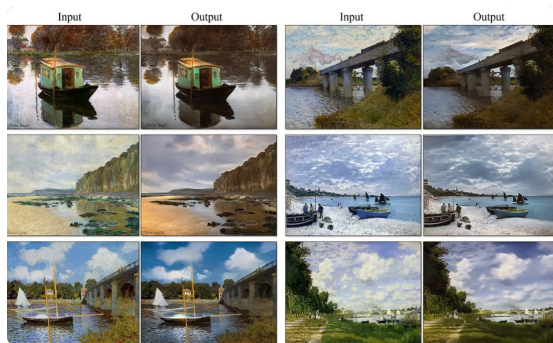
Redes Generativas Adversarias (GANs)

Mientras que las redes neuronales tradicionales buscan comprender o clasificar datos, los modelos generativos como las GANs buscan aprender la distribución de los datos para crear nuevos ejemplos similares.

Las GANs utilizan CNN tanto en el generador como en el discriminador para crear imágenes nuevas:

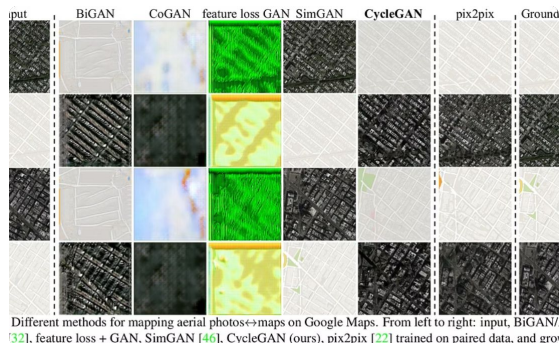
- El generador crea imágenes cada vez más realistas
- El discriminador intenta distinguir entre imágenes reales y generadas
- Ambas redes mejoran a través de un proceso adversarial

Aplicaciones de GANs



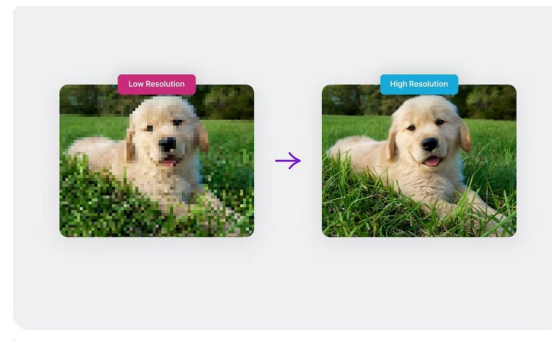
Arte Generativo

Creación de obras de arte originales basadas en estilos aprendidos.



Traducción de Imágenes

Convertir imágenes de un dominio a otro (día a noche, verano a invierno).



Super-resolución

Mejorar la resolución de imágenes de baja calidad.

Desafíos en el Entrenamiento de CNN

Desvanecimiento del Gradiente

En redes profundas, los gradientes pueden volverse muy pequeños, dificultando el entrenamiento de las primeras capas.

Sobreajuste

El modelo puede memorizar los datos de entrenamiento en lugar de aprender patrones generalizables.

Requisitos Computacionales

Las CNN profundas requieren hardware especializado (GPUs, TPUs) y mucho tiempo de entrenamiento.

Datos Limitados

Muchos problemas reales no tienen suficientes datos etiquetados para entrenar modelos desde cero.

Soluciones a los Desafíos

1

Inicialización de Pesos

Técnicas como la inicialización de He o Xavier ayudan a mantener gradientes estables.

2

Normalización por Lotes

Estabiliza y acelera el entrenamiento normalizando las activaciones en cada mini-lote.

3

Arquitecturas Residuales

Las conexiones de salto en ResNet permiten entrenar redes muy profundas sin degradación.

4

Aprendizaje por Transferencia

Utilizar modelos preentrenados reduce la necesidad de grandes conjuntos de datos.

Tendencias Futuras en CNN

Eficiencia Energética

Desarrollo de arquitecturas que requieren menos potencia computacional para funcionar en dispositivos con recursos limitados.

Aprendizaje Auto-Supervisado

Métodos que permiten a las redes aprender representaciones útiles sin necesidad de etiquetas manuales.

Arquitecturas Neuronales Híbridas

Combinación de CNN con otros tipos de redes como Transformers para aprovechar las fortalezas de cada una.

Consideraciones Éticas

El uso de CNN en aplicaciones del mundo real plantea importantes consideraciones éticas:

- Privacidad en sistemas de reconocimiento facial
- Sesgos en los datos de entrenamiento que pueden perpetuar discriminación
- Transparencia y explicabilidad de las decisiones tomadas por los modelos
- Impacto social y laboral de la automatización basada en visión artificial



Conclusión Final

Las redes neuronales convolutivas han revolucionado el campo de la visión por computadora y continúan evolucionando rápidamente. Dominar sus fundamentos te proporciona una base sólida para explorar aplicaciones más avanzadas y contribuir a este emocionante campo.

Este manual te ha proporcionado los conocimientos esenciales para comenzar tu viaje en el mundo de las CNN, desde la teoría básica hasta la implementación práctica. ¡El siguiente paso depende de ti!



Live Coding

¿En qué consistirá la Demo?

Mostraremos el flujo completo para optimizar e implementar un modelo predictivo CNN en Python para reconocimiento de imágenes.

1. Evaluación de una CNN usando matriz de confusión y métricas (Precision, Recall, F1).
2. Aplicar data augmentation para mejorar la robustez del modelo.
3. Implementar dropout y early stopping para evitar sobreajuste.
4. Introducir el uso de modelos preentrenados (ej. VGG16) mediante Transfer Learning.
5. Generar predicciones y analizar resultados.



Ejercicio N° 1

Optimiza e implementa tu CNN

Optimiza e implementa tu CNN

Contexto: 🙌

En este ejercicio consolidarás la implementación de un modelo CNN agregando técnicas avanzadas de optimización y validación.

Consigna: 📝

1. Toma la CNN que diseñaste en la Parte 1.
2. Evalúa el modelo con métricas avanzadas (Precision, Recall y F1).
3. Aplica al menos dos técnicas de optimización: dropout, data augmentation, early stopping o ajuste de hiperparámetros.
4. Implementa Transfer Learning con un modelo preentrenado y compara resultados.
5. Documenta los resultados obtenidos y los cambios en el rendimiento.

Paso a paso: ⚙️

1. Evalúa el modelo actual y registra sus métricas.
2. Aplica técnicas de optimización y vuelve a entrenar.
3. Prueba Transfer Learning con VGG16 o ResNet.
4. Compara el desempeño de los modelos y presenta conclusiones.

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





¡Ponte a prueba!

Momento de ejercitación

*Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.*

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< ¡Muchas gracias! >

